

01

Módulo 01 - OpenCV e Trabalho de Imagens

Visão de Computador e Stable Diffusion

Resumo do Módulo

- ◆ Instalar e validar OpenCV com pip para visão computacional.
- ◆ Ler, exibir e salvar imagens (imread, imshow, imwrite).
- ◆ Representação de imagens: pixéis, dimensões, canais e conversão de cores (BGR, RGB, Grayscale).
- ◆ Manipulação de imagens: redimensionar, rodar, inverter.
- ◆ Thresholding global e adaptativo (threshold, adaptiveThreshold).
- ◆ Filtragem e redução de ruído (Gaussian, median, bilateral).
- ◆ Detecção de bordas e contornos (Sobel, Scharr, Canny).
- ◆ Lab

Instalar OpenCV usando `pip` e validar a instalação

O **OpenCV** é uma biblioteca open-source para visão computacional e processamento de imagens.



<https://opencv.org/>

`pip install opencv-python` (core modules)

`pip install opencv-contrib-python` (extra modules)

```
C:\Users\rjamp>python
Python 3.11.0 (main, Oct 24 2022, 18:26:48) [MSC v.1933 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license" for more information.
>>> import cv2
>>> cv2.__version__
'4.9.0'
>>>
```

Ler, exibir e salvar imagens (imread, imshow, imwrite)

Primeiro:

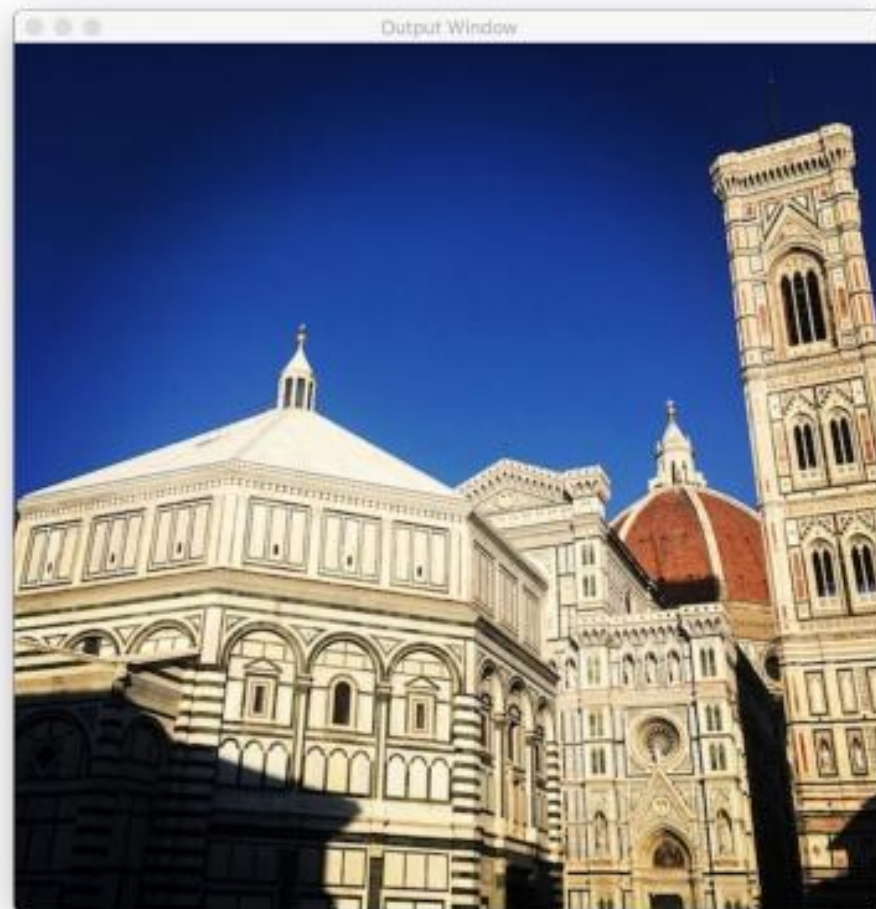
```
import cv2
```

#ou então o muito comum

```
import cv2 as cv
```

Depois:

ler uma imagem	- <code>cv2.imread()</code>
exibi-la	- <code>cv2.imshow()</code>
salvá-la	- <code>cv2.imwrite()</code>



```
import cv2
img = cv2.imread('duomo.jpg', 1)
cv2.imshow("Output Window", img)
cv2.waitKey()
```

A função `cv2.imread()`

`cv2.imread()` aceita 2 parâmetros:

1. nome do ficheiro
2. abordagem cromática:
 - 1** : Cor, negligenciando a transparência
 - 0** : Greyscale
 - 1**: Cor com o alpha channel



```
img = cv2.imread('duomo.jpg',1)
```

bgr		
0	0	0
0	0	0
0	255	255
255	0	255
255	255	0



```
img = cv2.imread('duomo.jpg',0)
```

gray		
0	127	255
255	127	0

A função `cv2.imshow()`

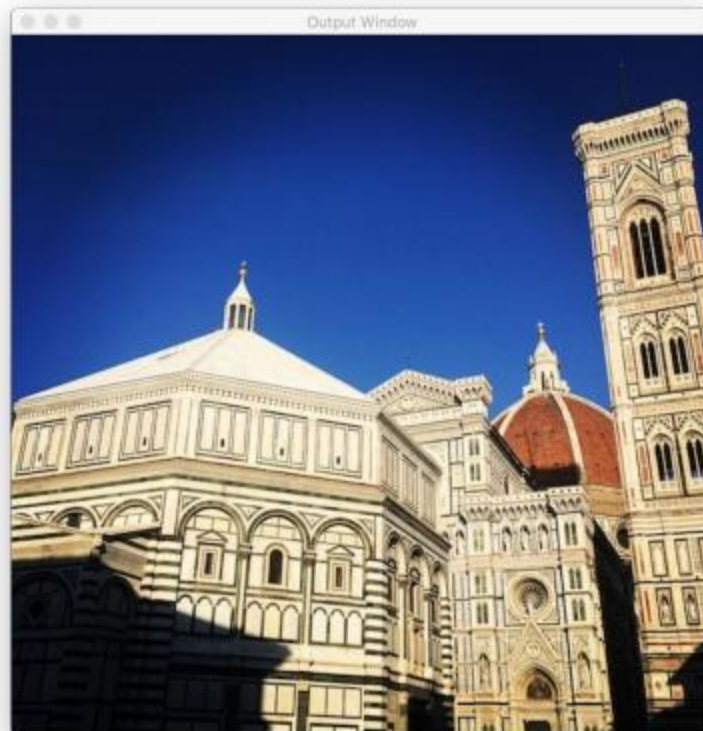
- ♦ É usada para exibir uma imagem numa janela.

`cv2.imshow("Output Window", img)`

Esta função aceita 2 parâmetros:

1. nome da janela de saída
2. A imagem a ser exibida na janela de saída

```
import cv2
img = cv2.imread('duomo.jpg',1)
img2 = cv2.imread('duomo.jpg',0)
cv2.imshow("Output Window", img)
cv2.imshow("ROI selector", img2)
cv2.waitKey()
cv2.destroyAllWindows()
```



A função `cv2.waitKey()`

`cv2.waitKey()`

é uma função de ligação do teclado

◆ Esta função retorna o valor ASCII da tecla pressionada

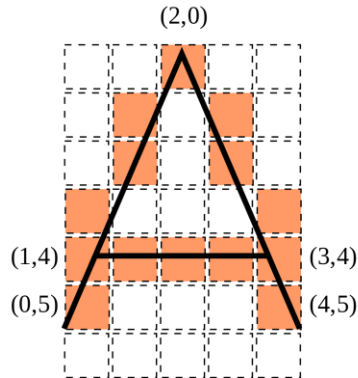
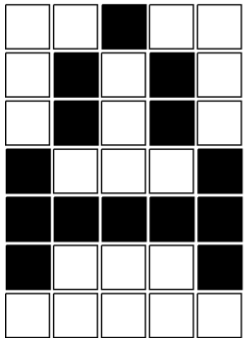
Tem 1 único parâmetro:

- ◆ **0 ou nenhum argumento**: aguarda indefinidamente a interrupção do teclado
- ◆ **Qualquer outro valor**: espera a duração desse valor em milisegundos

```
import cv2
img = cv2.imread('duomo.jpg', 1)
cv2.imshow("img", img)
cv2.waitKey()
```

Compreender a representação de imagens (pixéis, dimensões e canais)

♦Aceder a valores de pixéis e modificá-los



```
# Carregar a imagem em escala de cinza
imagem_gray = cv2.imread('exemplo.jpg', cv2.IMREAD_GRAYSCALE)

# Aceder um pixel (x=50, y=100)
valor_pixel = imagem_gray[100, 50]

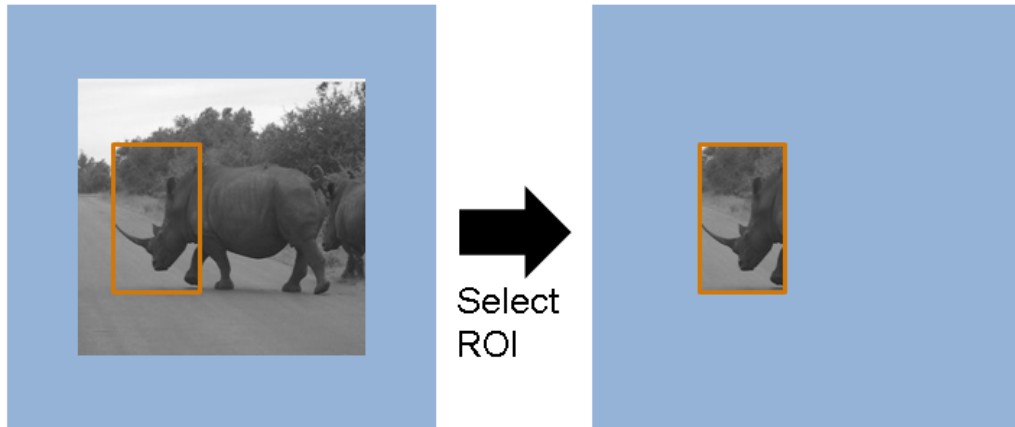
# Modificar o pixel para um tom de cinza específico (ex.: 200)
imagem_gray[100, 50] = 200
```

```
# Carregar a imagem com cor
imagem_bgr = cv2.imread('exemplo.jpg')

# Aceder um pixel (x=50, y=100)
pixel_bgr = imagem_bgr[100, 50]

# Modificar o pixel para azul puro (B=255, G=0, R=0)
imagem_bgr[100, 50] = [255, 0, 0]
```

Compreender a representação de imagens (pixéis, dimensões e canais)



▸ ROI – Região de Interesse

```
# Definir a ROI - Retângulo (x=50, y=50, largura=100, altura=100)
x, y, largura, altura = 50, 50, 100, 100

# Pintar a ROI com uma cor sólida (verde puro)
img[y:y+altura, x:x+largura] = [0, 255, 0]
```

◆ Aceder às propriedades da imagem (`img.shape`, `img.size`...)

Guardar imagens modificadas (imwrite)

cv2.imwrite(filename, img, params=None)

Parâmetro básicos:

1. nome do ficheiro
2. Imagem

Retorno: `True` ou `False`

para confirmar o sucesso da operação.

Parâmetros opcionais permitem ajustar qualidade (JPEG) ou compressão (PNG).

```
# Carregar a imagem
imagem = cv2.imread('exemplo.jpg')

# Definir uma ROI (Região de Interesse)
# - Exemplo: Retângulo (x=50, y=50, largura=100, altura=100)
roi = imagem[50:100, 50:100]

# Guardar apenas a ROI no disco
guardado = cv2.imwrite('roi.jpg', roi)
```

```
# Qualidade de 90% para o formato JPEG
cv2.imwrite('imagem.jpg', img, [cv2.IMWRITE_JPEG_QUALITY, 90])
```

```
# Compressão máxima
cv2.imwrite('imagem.png', img, [cv2.IMWRITE_PNG_COMPRESSION, 9])
```

Ex 01: Leitura e Exibição de Imagens com OpenCV

Ex 02: Carregar e Gravar uma Imagem com OpenCV

Ex 03: Leitura e Exibição de Imagens com Matplotlib

Ex 04: Manipulação de ROI (Região de Interesse) em Imagens

Funções de Desenho em OpenCV

- ♦OpenCV pode desenhar diferentes formas geométricas com estas funções:

`cv2.line()`

`cv2.circle()`

`cv2.rectangle()`

`cv2.ellipse()`

`cv2.putText()`

...

Operações aritméticas em Imagens

- ◆ Adição, subtração ou operação bit a bit.

`cv2.add()`

`cv2.addWeighted()`

`cv2.subtract()`

`cv2.bitwise_and()`

`cv2.bitwise_or()`

`cv2.bitwise_xor()`

`cv2.bitwise_not()`

Redimensionamento, rotação e inversão de imagens

- ◆Scaling

`cv2.resize()`

- ◆Translation

`cv2.warpAffine()`

- ◆Rotation

`cv2.getRotationMatrix2D()`

- ◆Affine Transformation

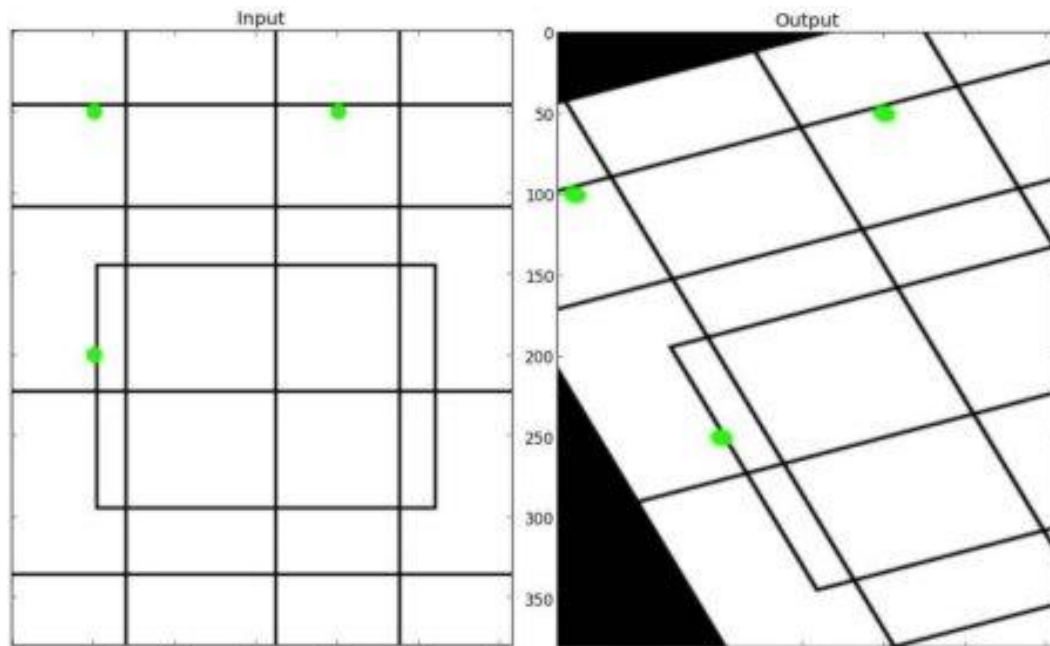
`cv2.getAffineTransform()` e `cv2.warpAffine()`

- ◆Perspective Transformation

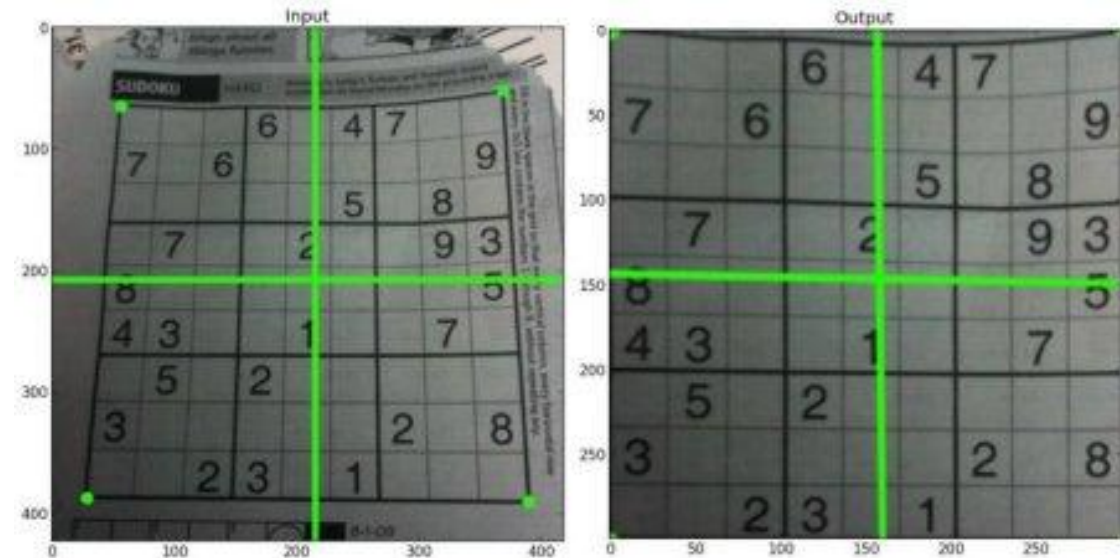
`cv2.getPerspectiveTransform()` e `cv2.warpPerspective()`

Affine Transformation e Perspective Transformation

Affine Transformation



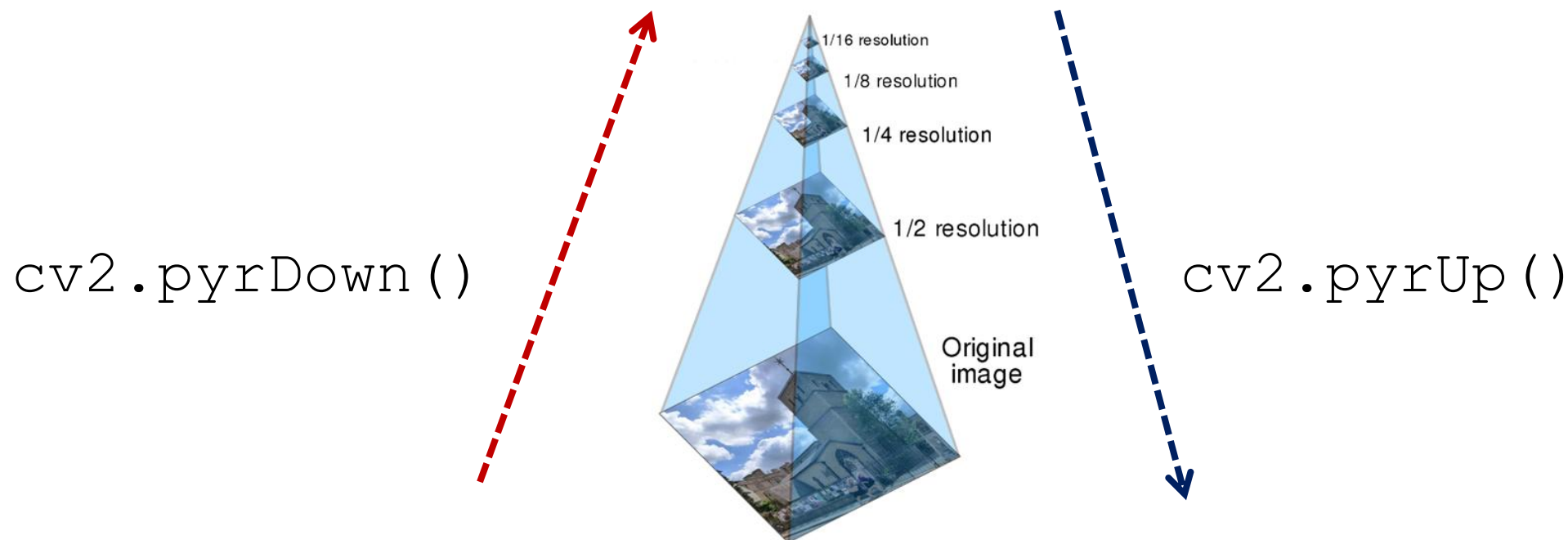
Perspective Transformation



Pirâmides de Imagem

Segmentação de imagens por pirâmides- **Gaussian** e **Laplacian**

- **Gaussian pyramid**: para reduzir a resolução de imagens (*downsampling*)
- **Laplacian pyramid**: para reconstruir uma imagem *upsampled* a partir de uma imagem mais baixa na pirâmide (com menos resolução)



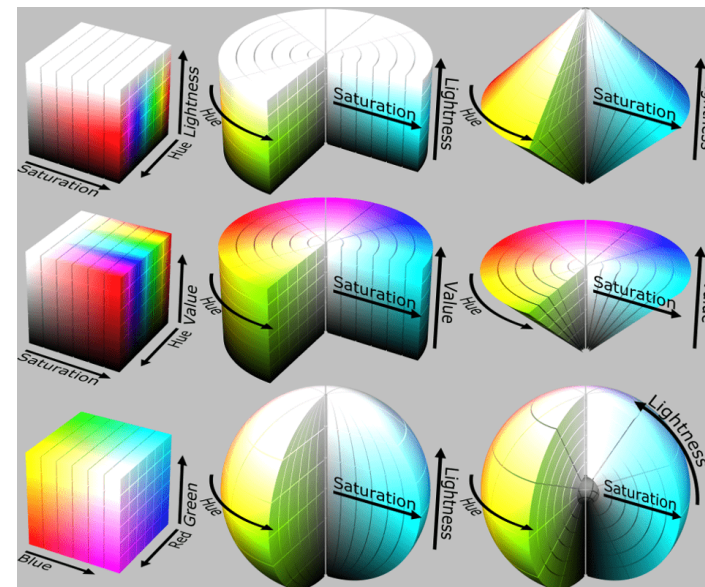
Conversão de espaços de cores das imagens (BGR, RGB, Grayscale)

No OpenCV também é possível converter imagens de um espaço de cores para outro, como:

BGR $\leftrightarrow$ Gray

BGR $\leftrightarrow$ HSV

etc



Existem mais de **150** métodos de conversão de espaço de cores disponíveis no OpenCV.

```
cv2.cvtColor()
```

```
cv2.inRange()
```

...

Alterar o espaço de cores é o método mais simples na detecção de objetos

1. Tirar cada frame do vídeo



2. Converter de BGR para HSV espaço de cores



3. Limitar a imagem HSV para um intervalo de cores azuis



4. Extrair o objeto azul sozinho e obter a nova posição

Ex 05: Funções de Desenho no OpenCV

Ex 06: Operações Aritméticas e Máscaras de Imagens no OpenCV

Ex 07: Transformações Geométricas no OpenCV

Ex 08: Carregamento e Correção de Cores (BGR para RGB e HSV)

Thresholding de imagens e as suas aplicações

- ◆ Thresholding é um método de segmentação de imagem.
- ◆ **Simple thresholding** - Se o valor do pixel for *maior do que um valor limite*, é *atribuído um valor* (pode ser branco), **caso contrário**, é **atribuído outro valor** (pode ser preto).



`cv2.threshold()`

Thresholding adaptativo no OpenCV

- ♦ **Adaptive Thresholding** - o algoritmo calcula o limite para uma pequena região da imagem.

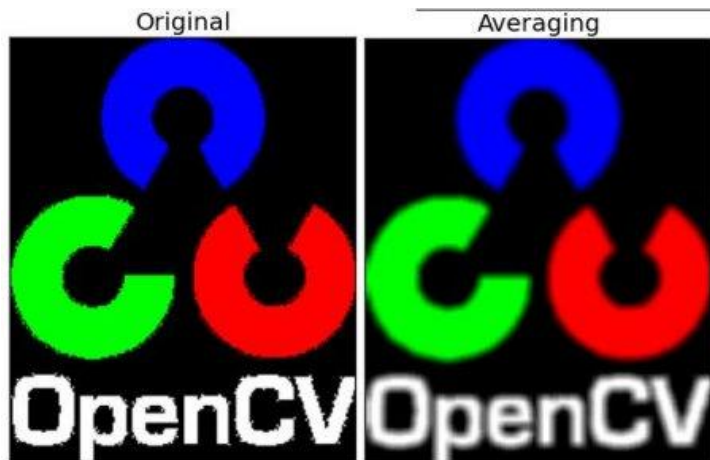
... obtemos limiares diferentes para diferentes regiões.



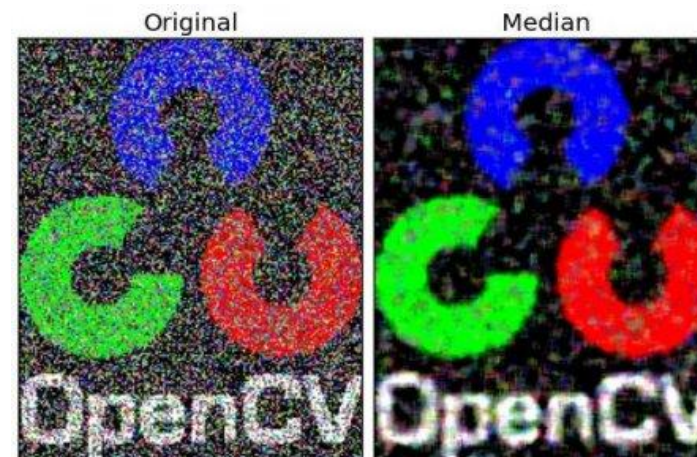
```
cv2.adaptiveThreshold()
```

Filtros comuns no OpenCV: BLUR Gaussian, median e bilateral

- ♦ Averaging – `cv2.blur()`



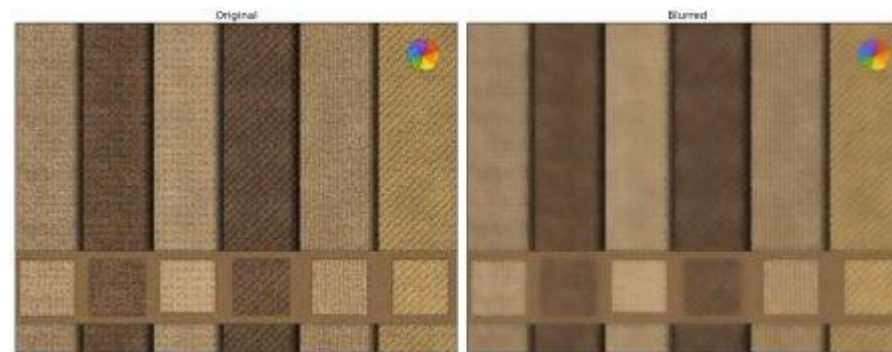
- ♦ Median – `cv2.medianBlur()`



- ♦ Gaussian – `cv2.GaussianBlur()`



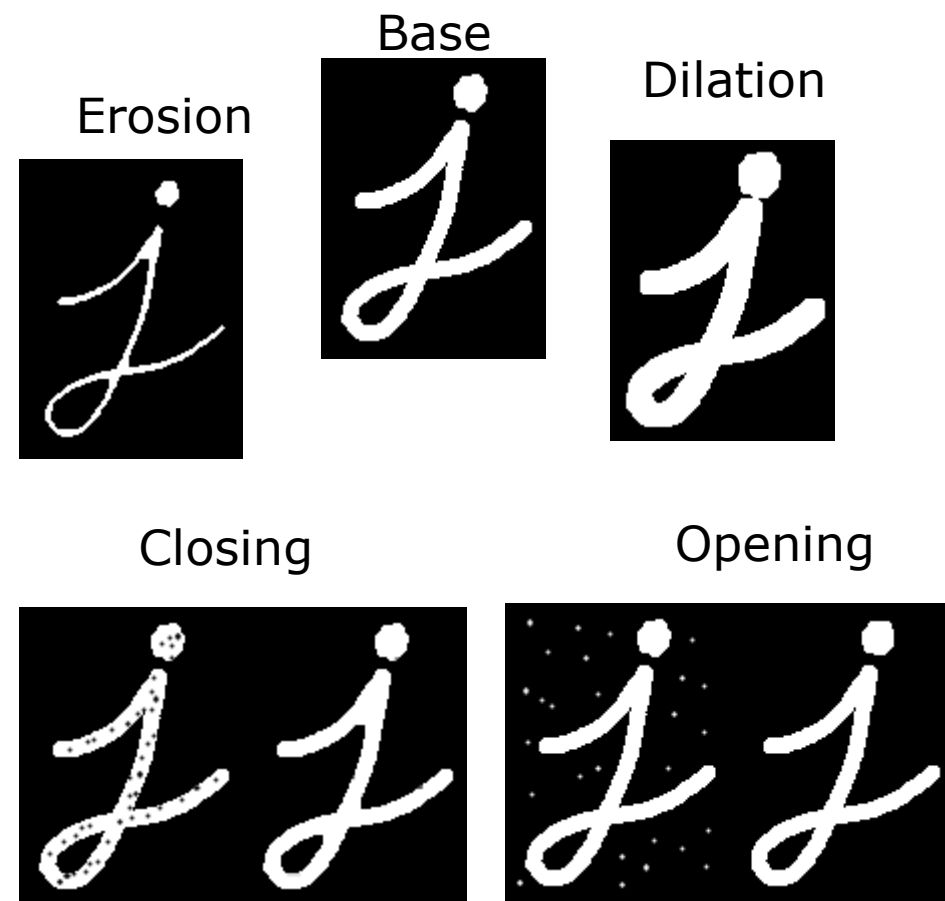
- ♦ Bilateral – `cv2.bilateralFilter()`



Transformações Morfológicas

`cv2.erode()`, `cv2.dilate()`, `cv2.morphologyEx()` ...

- ♦ **Erosion**: Objeto reduzido, removendo bordas externas.
- ♦ **Dilation**: Objeto expandido, preenchendo lacunas.
- ♦ **Opening**: Ruídos pontuais removidos, mantendo a forma do objeto.
- ♦ **Closing**: Pequenos buracos preenchidos dentro do objeto.
- ♦ **Gradient**: Contornos realçados, mostrando a diferença entre dilatação e erosão.
- ♦ **Top-Hat**: Detalhes claros realçados sobre o fundo.
- ♦ **Black-Hat**: Detalhes escuros realçados sobre o fundo.



Transformações Morfológicas

Image I



Erosion $I \ominus B$



Dilatation $I \oplus B$



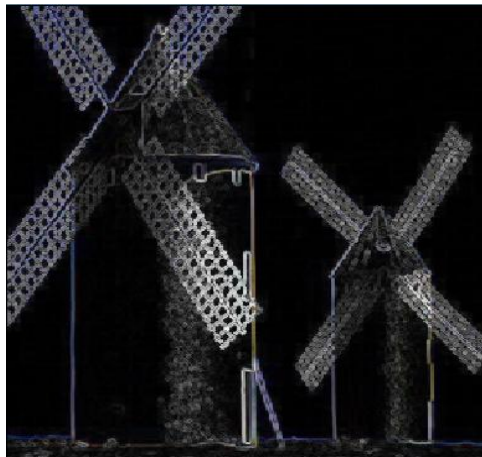
Opening $I \circ B = (I \ominus B) \oplus B$



Closing $I \bullet B = (I \oplus B) \ominus B$



Grad(I) = $(I \oplus B) - (I \ominus B)$



TopHat(I) = $I - (I \ominus B)$



BlackHat(I) = $(I \oplus B) - I$



Extração do Foreground (primeiro plano) usando o algoritmo GrabCut

`cv2.grabCut()`



Ex 09: Comparação entre Simple Thresholding e Adaptive Thresholding

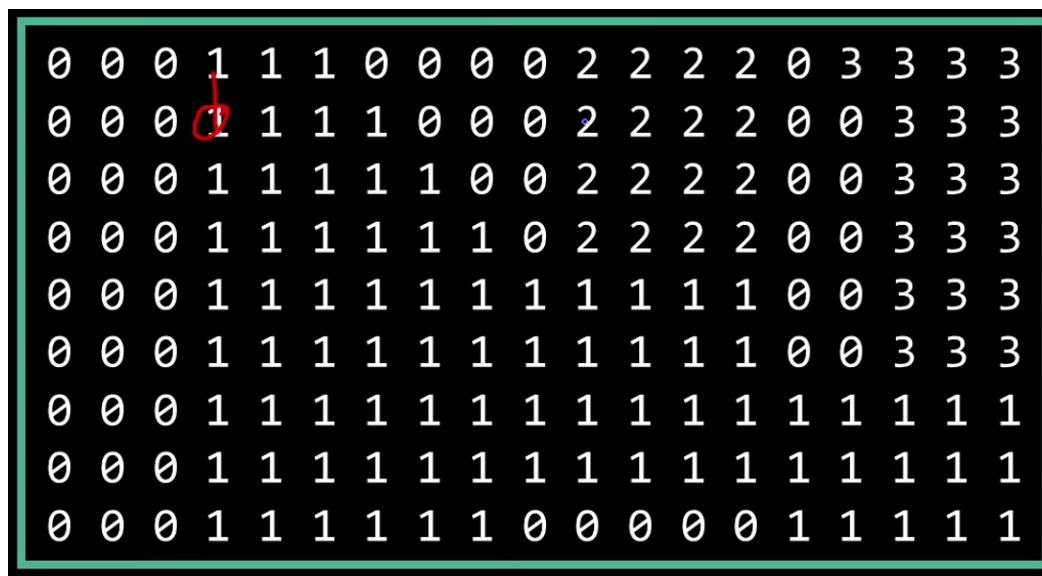
Ex 10: Técnicas de Suavização de Imagens (Smoothing/Blurring)

Ex 11: Operações Morfológicas com OpenCV

Ex 12: Extração do Primeiro Plano com GrabCut

Contornos em OpenCV

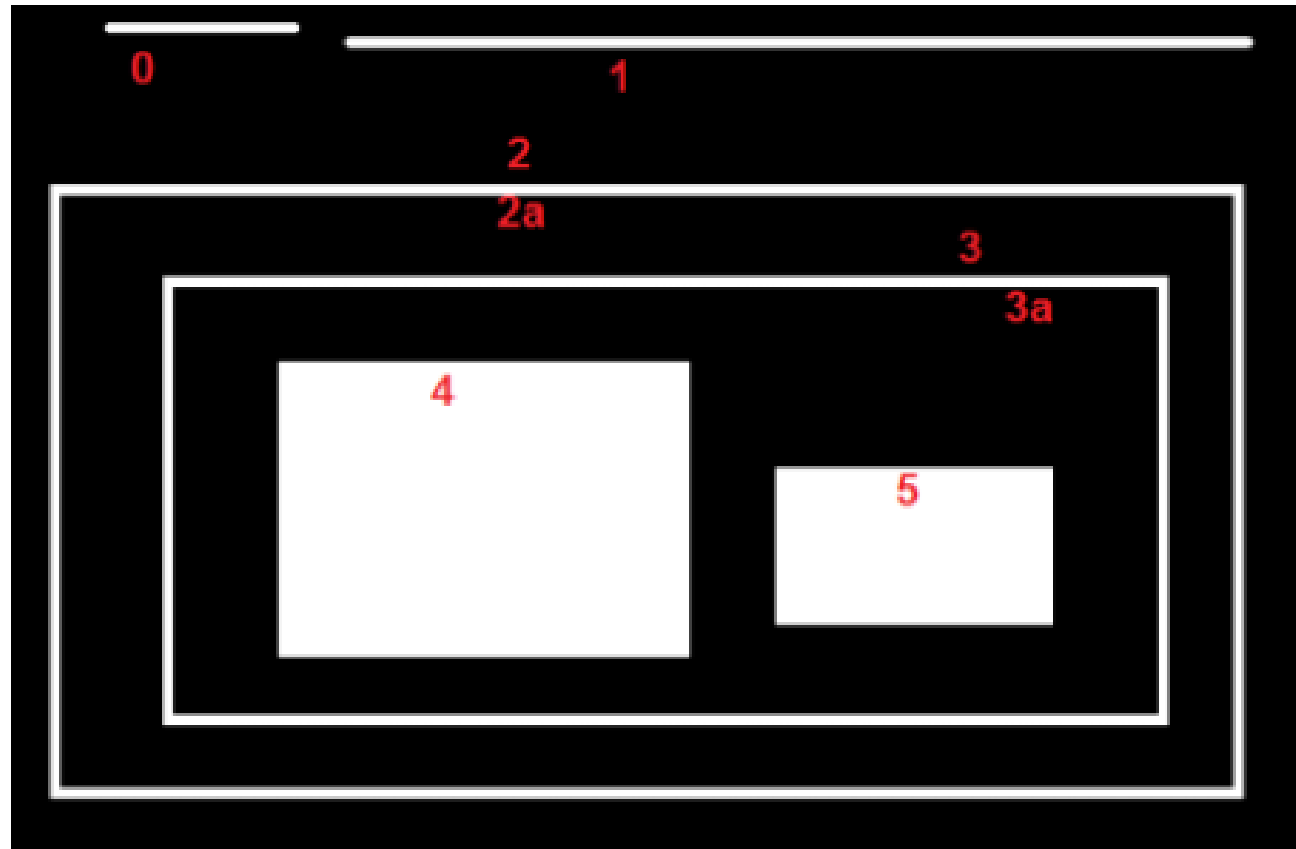
- ♦ Os *contornos* podem ser explicados simplesmente como uma *curva que une todos os pontos contínuos* (ao longo do limite), tendo a mesma cor ou intensidade.



- ♦ Os contornos são uma ferramenta útil para análise de formas e **deteção e reconhecimento de objetos**.

Componentes conectados = find contours

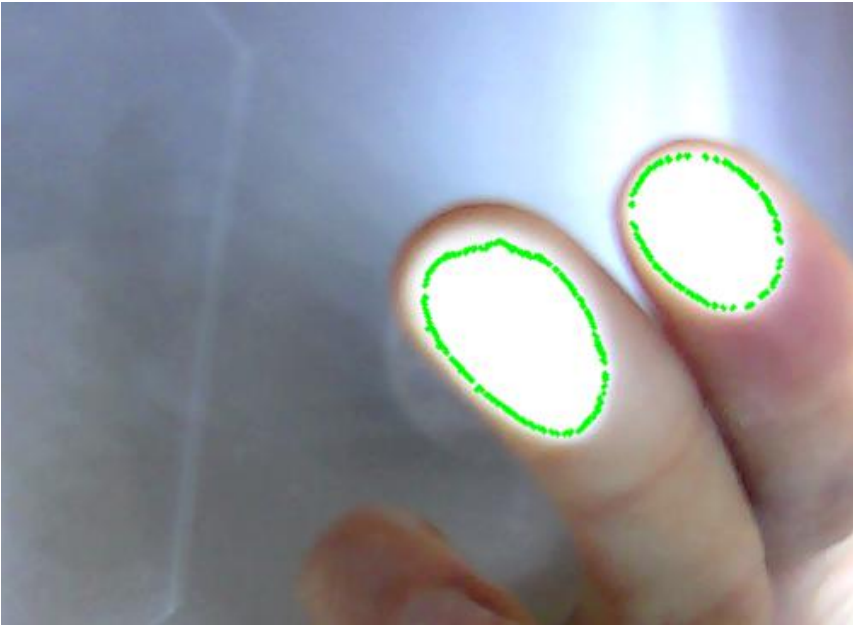
```
contours, hierarchy = cv2.findContours()
```



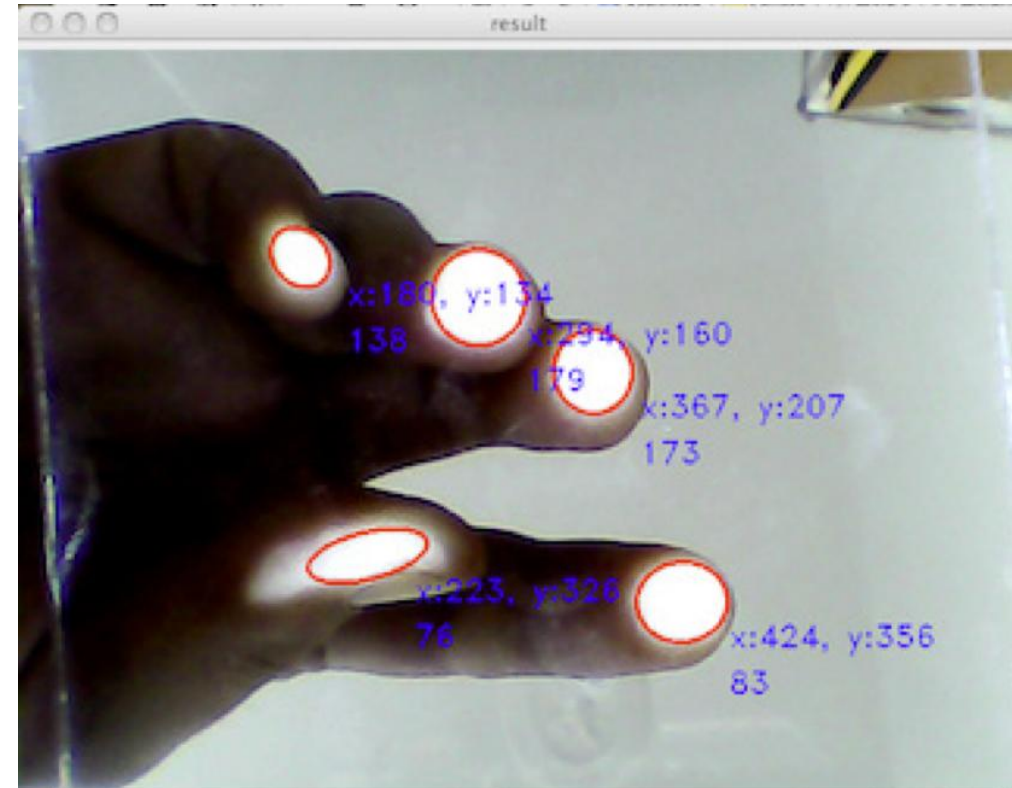
Encontrar o centro de um contorno

`cv2.minEnclosingCircle(contour)`

círculo de ajuste = centro, raio



contorno = caminho dos pontos



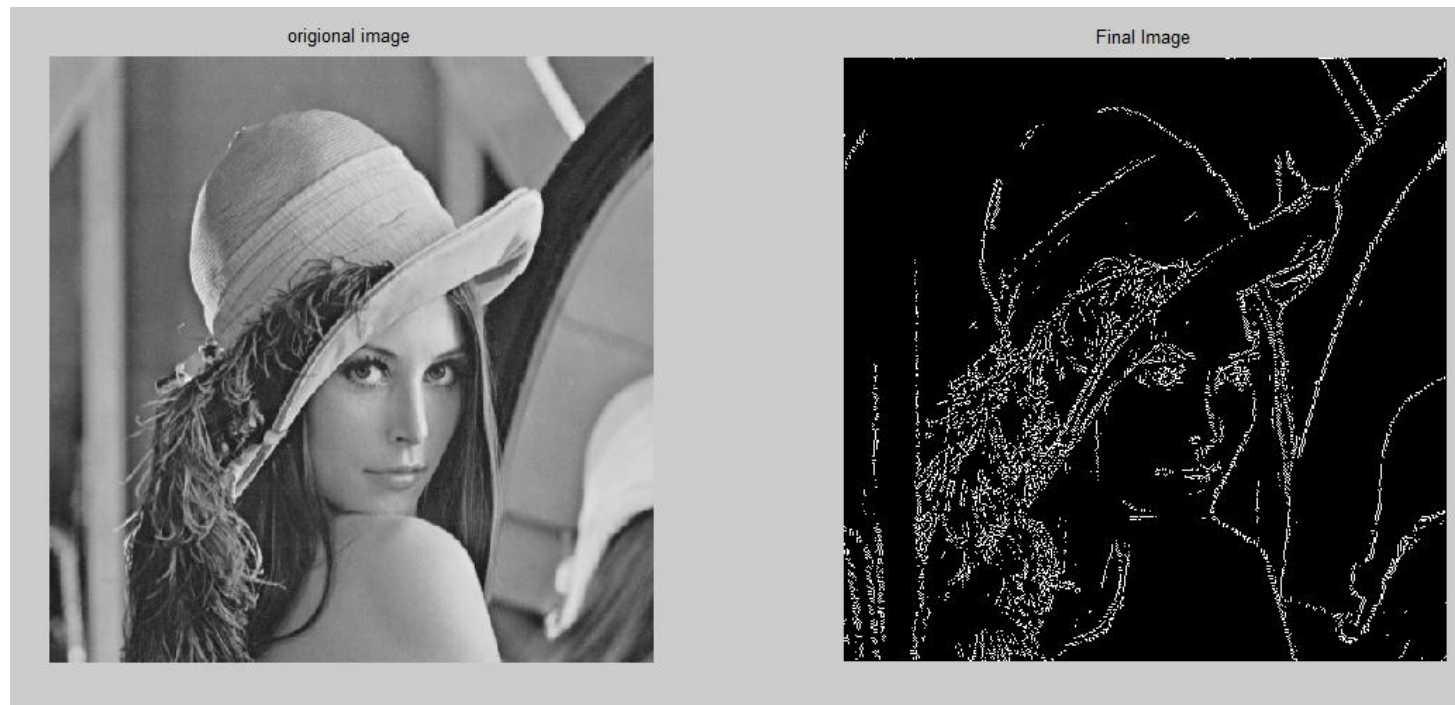
```
cv2.circle(frame, draw_center, draw_radius, (0,255,0), 2)
```

```
cv2.putText(frame, textstring, (int(x+50), int(y)),  
            cv2.FONT_HERSHEY_SIMPLEX, 0.5, (255,255,255), 1)
```

Deteção Canny Edge

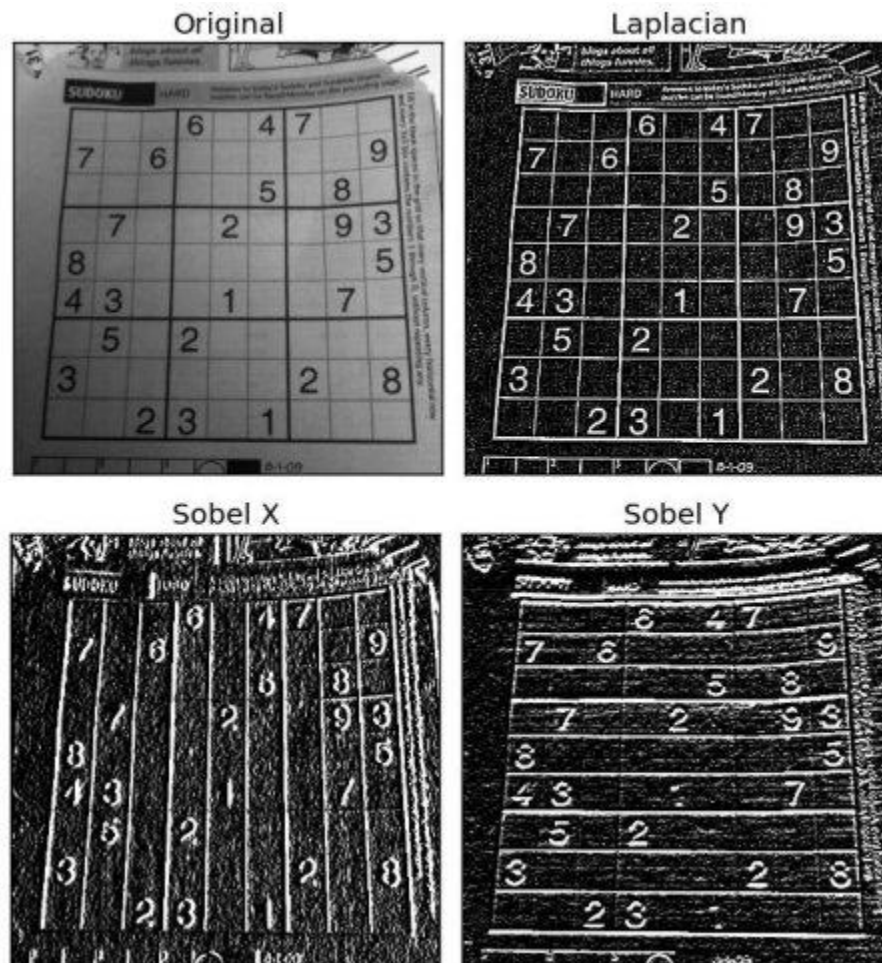
- ♦ Canny Edge Detection é um algoritmo popular de detecção de borda.
- Foi desenvolvido por John F. Canny em 1986.

`cv2.Canny()`



Gradientes de imagem

◆ `cv2.Sobel()` , `cv2.Laplacian()`

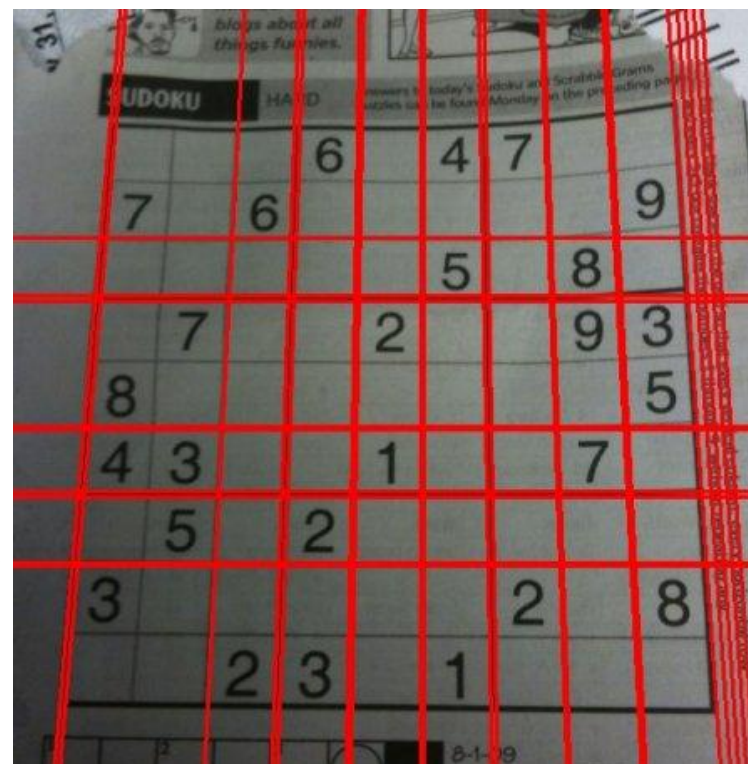


Hough Line Transform

- ♦ A transformada de Hough é uma ferramenta incrível que permite identificar linhas e outras formas (círculos, ângulos, etc).

- `cv2.HoughLines()`

- `cv2.HoughLinesP()`



Hough Circle Transform

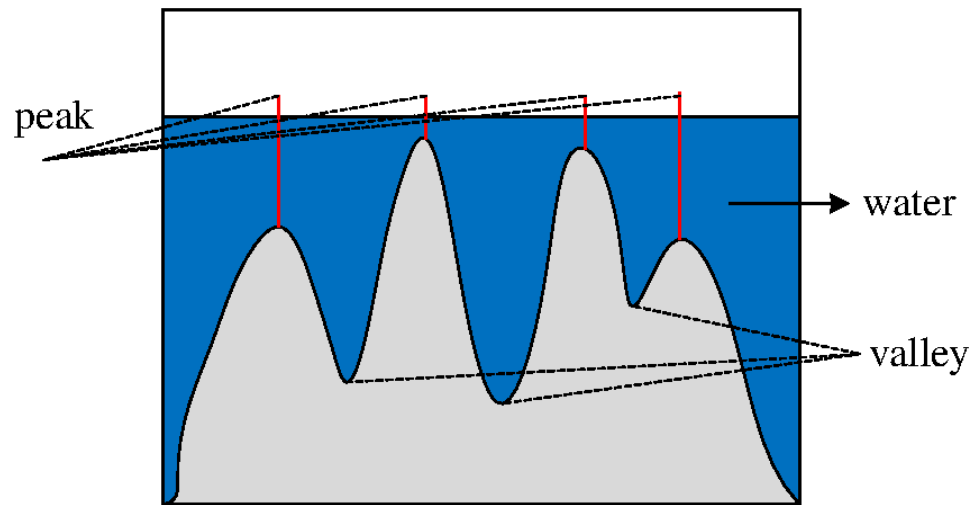
- ◆ Usar Hough Transform para localizar círculos em uma imagem

```
cv2.HoughCircles()
```



Segmentação de Imagem com Watershed Algorithm

- ◆ Em geologia, uma bacia hidrográfica (**Watershed**) é uma divisão que separa bacias hidrográficas adjacentes

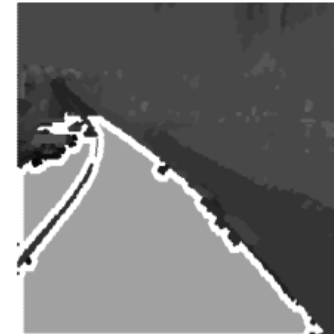
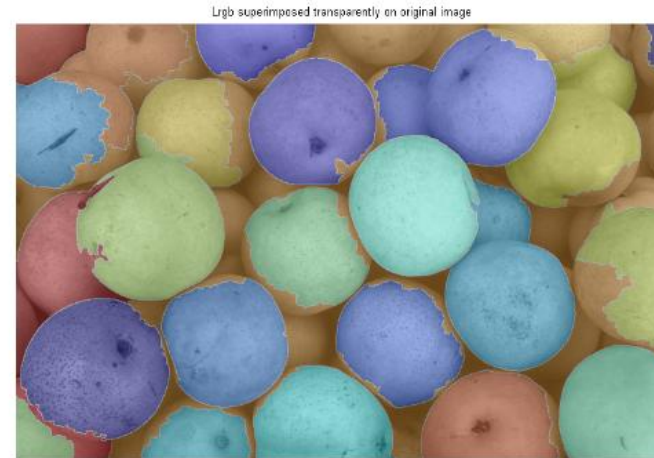


Segmentação de Imagem com Watershed Algorithm

`cv2.watershed()`



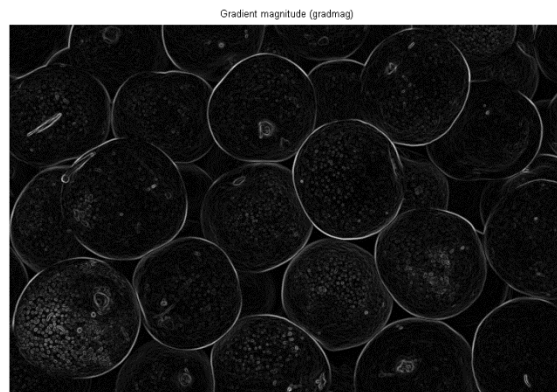
Image courtesy of Corel®



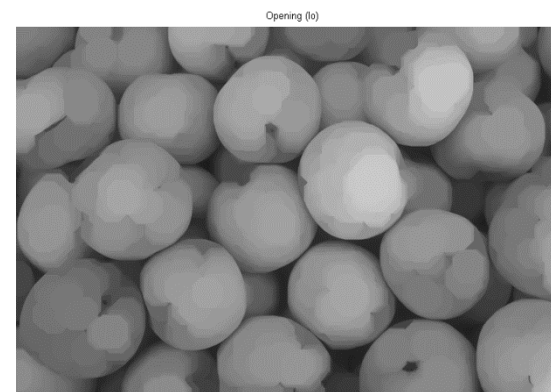
Processo de transformação Watershed



Source: Uma
imagem em escala
de cinza

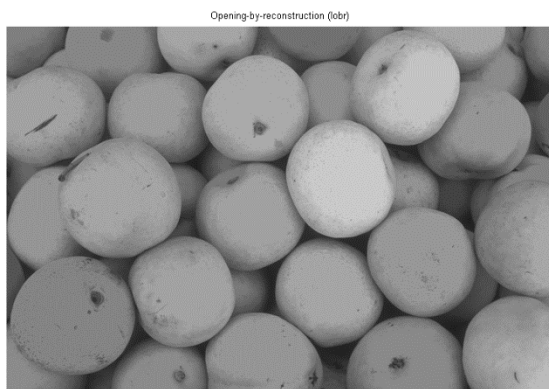


Step 1: Usar a
magnitude do gradiente
como a função de
segmentação - O
gradiente é alto nas
bordas dos objetos e
baixo (principalmente)
dentro dos objetos.

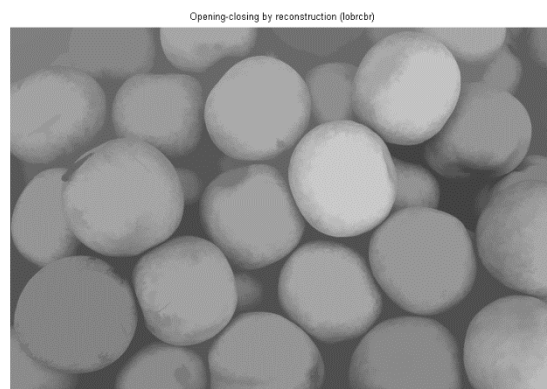


Step 2: Marcar os
objetos em
primeiro plano

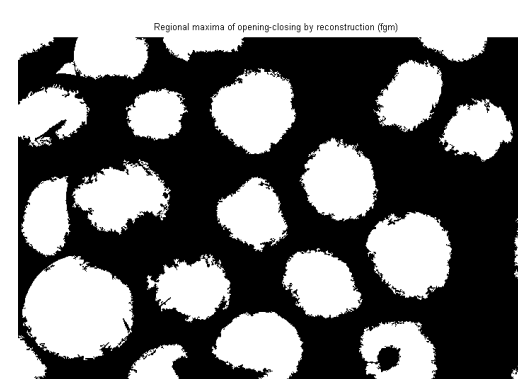
Processo de transformação Watershed



Step 3: Calcular a abertura por reconstrução da imagem

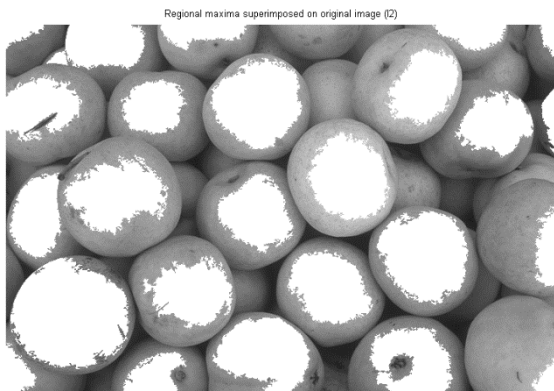


Step 4: Seguir a abertura com um fecho pode remover as manchas escuras e marcas do caule.

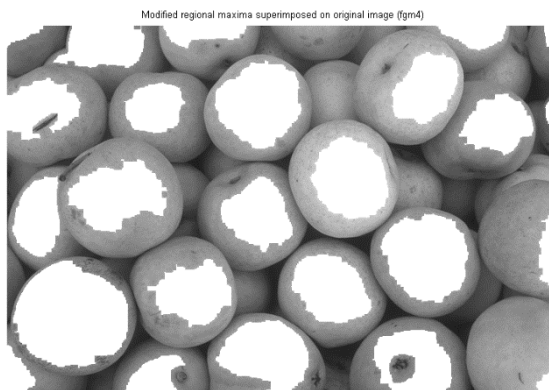


Step 5: Calcular os máximos regionais para obter bons marcadores de primeiro plano.

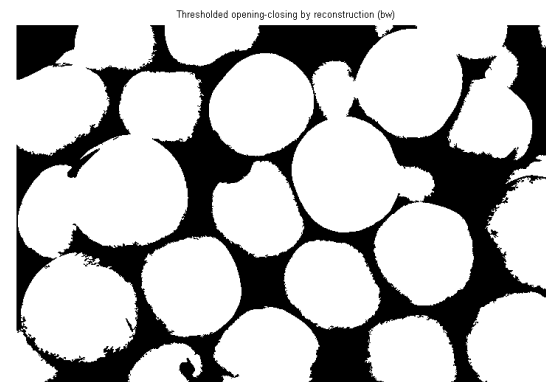
Processo de transformação Watershed



Step 6: Sobrepor a imagem do marcador de primeiro plano à imagem original, Observe que os marcadores de primeiro plano em alguns objetos vão até a borda dos objetos

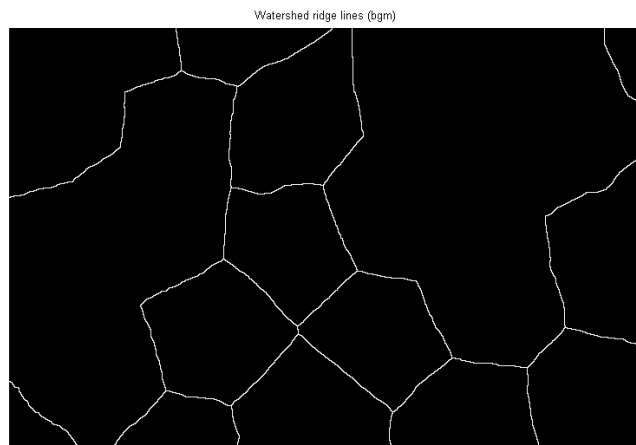


Step 7: limpar as bordas das bolhas de marcação e, em seguida, encolhe-as um pouco

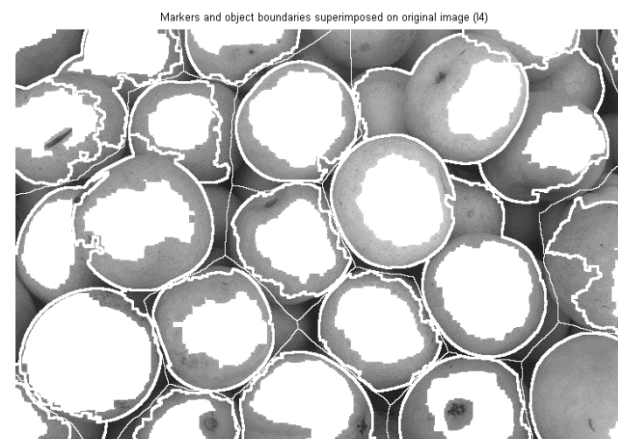


Step 8: Calcular marcadores de plano de fundo, começando com a operação de limite

Processo de transformação Watershed

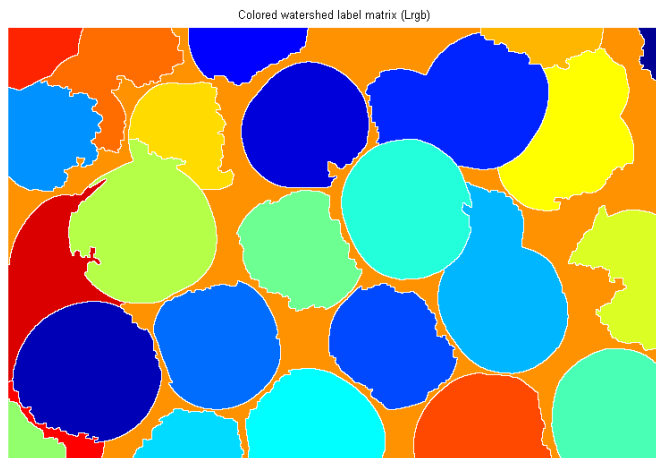


Step 9: Calcular marcadores de fundo, usando a transformada de bacia hidrográfica da transformada de distância e, em seguida, procurando as linhas de crista da bacia hidrográfica do resultado

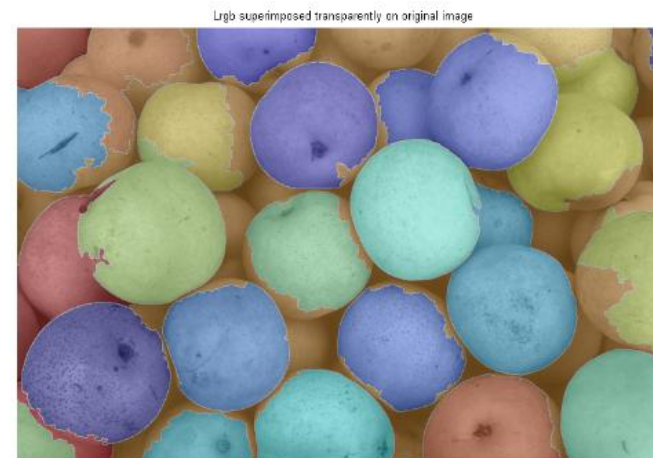


Step 10: Visualizar o resultado, uma das técnicas é sobrepor os marcadores de primeiro plano, marcadores de plano de fundo e limites de objetos segmentados.

Processo de transformação Watershed



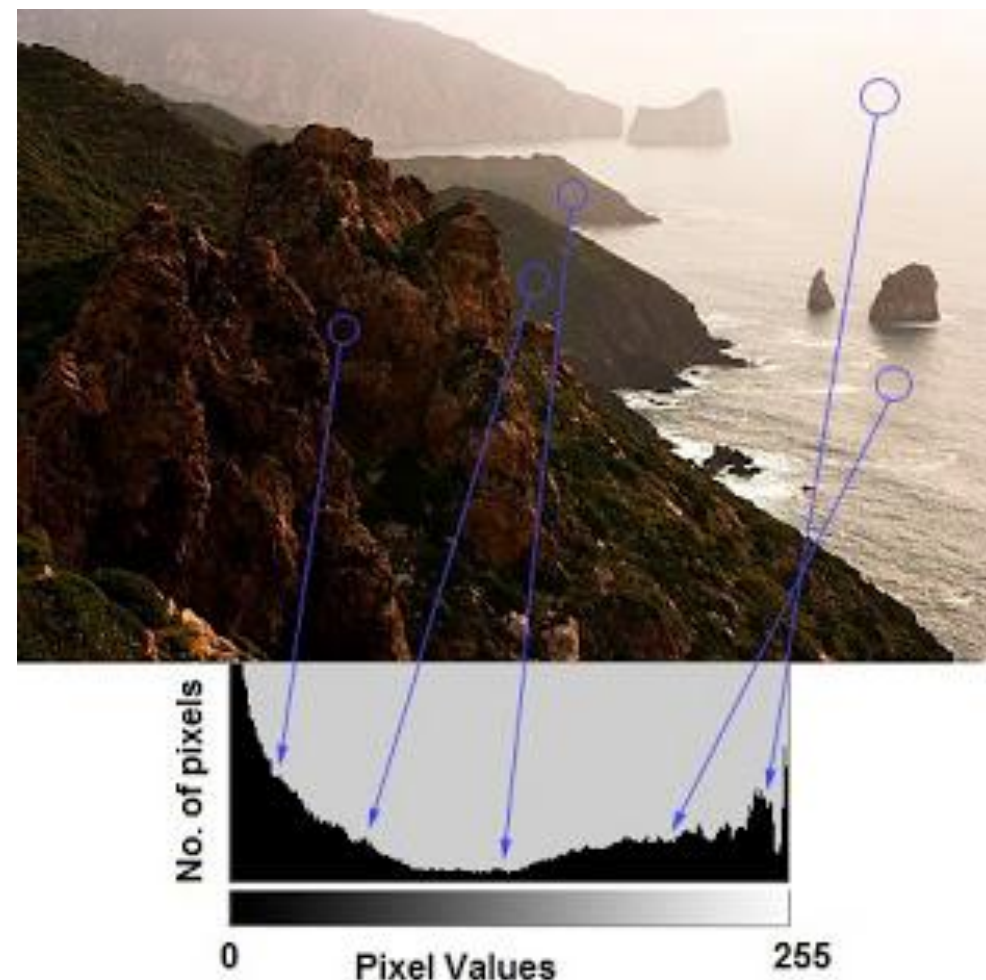
*Outra técnica de visualização útil é exibir a matriz de rótulos como uma imagem colorida



Podemos usar a transparência para sobrepor esta matriz de rótulo pseudo-cor sobre a imagem de intensidade original.

Histogramas em OpenCV

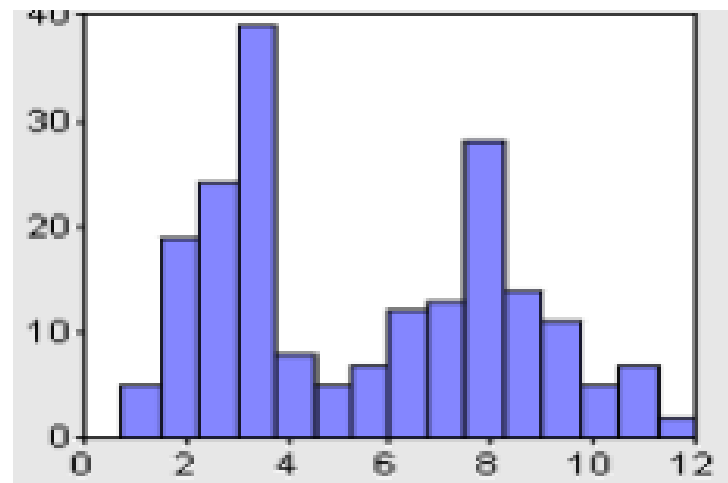
- ♦ *Histograma* é um gráfico ou gráfico, que lhe dá uma idéia geral sobre a *distribuição de intensidade de uma imagem*.
 - ♦ É um gráfico com valores de pixel (variando por vezes no maximo de 0 a 255) no eixo X e número correspondente de pixels na imagem no eixo Y.
- `cv2.calcHist()`
 - `np.histogram()`



Histogramas em OpenCV



Vamos considerar
esta imagem

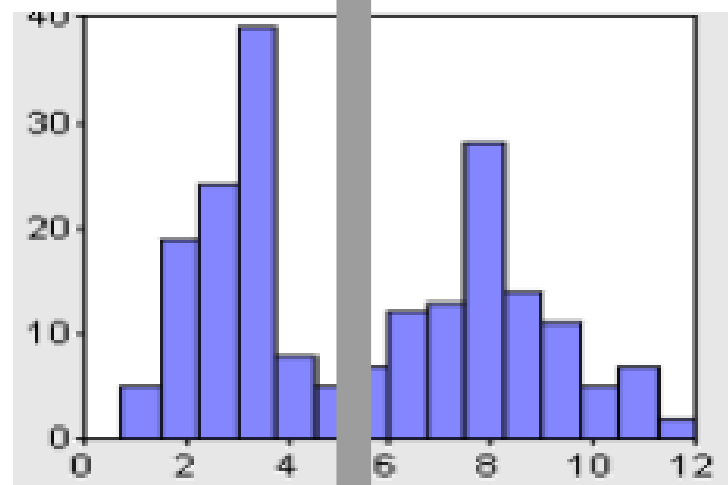


como
o histograma funciona?

preto

branco

Histogramas em OpenCV



preto

branco

onde colocaria o limiar
para encontrar todas as árvores?

Template Matching

- ♦ **Template Matching** é um método para pesquisar e encontrar a localização de uma imagem de modelo em uma imagem maior.

```
cv2.matchTemplate()
```



Transformadas de imagem no OpenCV

- ◆ Transformada de *Fourier* e *Inversa de Fourier* de imagens no OpenCV

`cv2.dft()` , `cv2.idft()`

- ◆ Existem funções FFT disponíveis no Numpy

Ex 13: Análise da Imagem com DFT e IDFT

Ex 14: Análise de Histograma de Imagem em Tons de Cinza

Ex 15: Detecção de Bordas com Canny, Sobel e Laplacian

Ex 16: Segmentação Watershed, Detecção de Linhas e Círculos (Hough
Lines e Hough Circles)

Ex 17: Detecção de Objetos com Template Matching